≈ ≈ ≈ ≈

TideWatch

Technology Stack & Run Guide

A comprehensive breakdown of the backend and frontend technologies powering TideWatch — an ML-based Network Intrusion Detection System (NIDS) dashboard built on the CICIDS2017 benchmark — along with the exact commands needed to install, train, migrate, and run the project.

Project: **E:\TideWatch** • Generated: October 20, 2026 • Version: 1.0.0

Table of Contents

1. Project Overview
2. High-Level Architecture
3. Backend Technologies (Python / FastAPI)
4. Frontend Technologies (React / Vite)
5. How Frontend & Backend Connect
6. Commands to Run the Project
7. API Endpoint Reference
8. Environment Variables

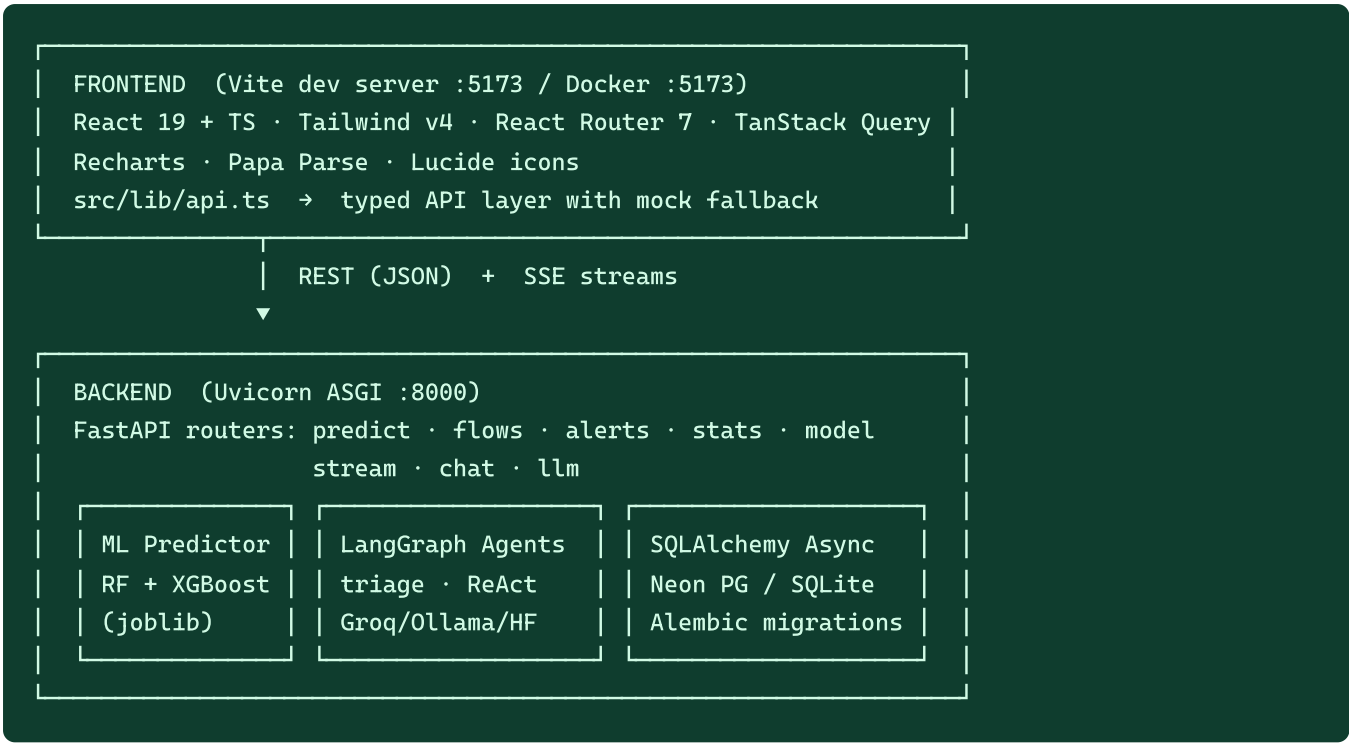
1. Project Overview

TideWatch is a full-stack, ML-powered Network Intrusion Detection System. It inspects bidirectional network flow telemetry in real time, evaluating statistical packet arrival patterns, inter-arrival intervals, and bidirectional window sizes calibrated on the **CICIDS2017** cybersecurity benchmark — rather than relying on static signatures or encrypted payload inspection.

> **Monorepo layout:** a single repository containing two independently runnable applications — a **Python FastAPI backend** (`app/`) and a **React + TypeScript frontend** (`src/`) — plus an ML training pipeline (`ml/`), Alembic migrations (`alembic/`), tests (`tests/`), and Docker orchestration.

Aspect	Detail
Application type	Full-stack ML Network Intrusion Detection System (NIDS) dashboard
Backend	Python 3.11 · FastAPI · SQLAlchemy (async) · scikit-learn / XGBoost · LangGraph agents
Frontend	React 19 · TypeScript · Vite 8 · Tailwind CSS v4 · React Router 7 · TanStack Query 5
Database	Neon PostgreSQL (production) or SQLite (local dev), both via async drivers
AI / LLM layer	Groq, Ollama, or HuggingFace providers with automatic fallback + rule-based heuristics
Real-time	Server-Sent Events (SSE) streams for live traffic replay and token-streaming chat
Containerization	Docker & Docker Compose (API + optional Ollama + frontend services)

2. High-Level Architecture


FRONTEND (Vite dev server :5173 / Docker :5173)
React 19 + TS · Tailwind v4 · React Router 7 · TanStack Query
Recharts · Papa Parse · Lucide icons
src/lib/api.ts → typed API layer with mock fallback
REST (JSON) + SSE streams
BACKEND (Uvicorn ASGI :8000)
FastAPI routers: predict · flows · alerts · stats · model
stream · chat · llm
ML Predictor
RF + XGBoost
(joblib)
LangGraph Agents
triage · ReAct
Groq/Ollama/HF
SQLAlchemy Async
Neon PG / SQLite
Alembic migrations

BACKEND

3. Backend Technologies

The backend is a **Python 3.11** asynchronous API service built on **FastAPI**, combining a machine-learning inference engine, a LangGraph agentic AI layer, and an async ORM with dual PostgreSQL/SQLite support.

3.1 CORE WEB FRAMEWORK

Technology	Version	Role in TideWatch
Python	3.11	Runtime; Docker base image `python:3.11-slim`
FastAPI	≥ 0.115	Async web framework; auto OpenAPI docs at `/docs`, DI, routers
Uvicorn	≥ 0.32 (standard)	ASGI server running the app (`--reload` in dev, 2 workers in Docker)
Pydantic	v2 (≥ 2.9)	Request/response schemas in `app/schemas/`; validation & serialization
pydantic-settings	≥ 2.6	Typed `Settings` class loading `.env` (`app/core/config.py`)
sse-starlette	≥ 2.1	Server-Sent Events responses for live traffic & streaming chat
python-multipart	≥ 0.0.12	multipart file uploads (CSV batch classification)
httpx	≥ 0.27	Async HTTP client (LLM provider calls, tests)

3.2 DATA PERSISTENCE & MIGRATIONS

Technology	Version	Role in TideWatch
SQLAlchemy	2.0 async (≥ 2.0.35)	Async ORM — models for flows, alerts, AI reports, chat, model runs
asyncpg	≥ 0.30	Async driver for production **PostgreSQL / Neon**
aiosqlite	≥ 0.20	Async driver for zero-setup local **SQLite** dev database
Alembic	≥ 1.13	Schema migrations (`alembic upgrade head`); initial schema in `001_initial_schema.py`

> **Dual-database support:** `Settings.async_database_url` auto-normalizes the `DATABASE_URL` — rewriting `postgres://` → `postgresql+asyncpg://` and `sqlite://` → `sqlite+aiosqlite://`, so either Neon cloud or local SQLite works with one config value.

3.3 MACHINE LEARNING ENGINE

Technology	Version	Role in TideWatch
scikit-learn	≥ 1.5	RandomForest classifier, scalers, metrics (precision/recall/F1/ROC-AUC)
XGBoost	≥ 2.1	Second ensemble member; RandomForest + XGBoost voting
imbalanced-learn	≥ 0.12	Class-imbalance handling on CICIDS2017 attack distributions
pandas / numpy	≥ 2.2 / 1.26 (<2.0)	Feature engineering over 78-dimensional flow vectors; CSV I/O
joblib	≥ 1.4	Serialization of `model.joblib`, `scaler.joblib`, `label_encoder.joblib`

The training pipeline lives in `ml/train.py` and emits artifacts to `artifacts/` (model, scaler, label encoder, `features.json`, `metrics.json`, `test_samples.csv`). If artifacts are missing, the API **trains automatically at startup**. The singleton predictor (`app/ml/predictor.py`) validates input columns and serves single + batch inference.

3.4 AGENTIC AI LAYER (LLM)

Technology	Version	Role in TideWatch
LangChain / langchain-core	≥ 0.3	LLM abstractions, prompts, output handling
LangGraph	≥ 0.2.20	Stateful triage workflow: classify → enrich → explain → mitigate → score → persist
langchain-groq	≥ 0.2	Cloud LLM (default: `llama-3.3-70b-versatile`)
langchain-ollama	≥ 0.2	Local LLM option (`llama3.1:8b`), Docker profile included
langchain-huggingface	≥ 0.1	HuggingFace option (`Qwen/Qwen2.5-7B-Instruct`)

> **Resilience:** the backend cascades through configured providers on failure, and `app/agents/fallbacks.py` provides pure **rule-based triage heuristics** so the API remains fully functional with *no LLM API key at all*. A ReAct chat agent (`chat_agent.py`) streams tokens over SSE with safe ORM tools (`query_flows`, `summarize_window`, `knowledge`).

3.5 QUALITY, CONFIG & OPS

Technology	Version	Role in TideWatch
pytest + pytest-asyncio	≥ 8.3 / ≥ 0.24	Unit & integration tests against in-memory async SQLite
pytest-mock	≥ 3.14	Mocking in predictor/agent tests
python-dotenv	≥ 1.0	Loads `.env` configuration
Structured logging	stdlib	JSON logging module in `app/core/logging.py`
Docker / Compose	Compose 3.9	API container (trains model at build) + optional Ollama + frontend services

Backend Folder Map

```
app/
├── main.py           # FastAPI entrypoint: lifespan, CORS, health, routers
├── api/routes/       # predict · flows · alerts · stats · model · stream · chat · llm
├── agents/           # LangGraph triage, ReAct chat agent, fallbacks, tools, state
├── ml/predictor.py   # Singleton inference engine (loads joblib artifacts)
├── models/           # SQLAlchemy ORM: flow, alert, ai_report, chat, model_run
├── schemas/          # Pydantic v2 request/response contracts (camelCase JSON)
├── db/               # async engine + session (PostgreSQL or SQLite)
└── core/             # pydantic-settings config + structured logging
ml/train.py           # RandomForest + XGBoost training pipeline
alembic/              # migrations (001_initial_schema)
tests/                # pytest: predictor, batch endpoint, agent
```

FRONTEND

4. Frontend Technologies

The frontend is a modern **single-page application** written in **React 19 + TypeScript**, built with **Vite 8** and styled with **Tailwind CSS v4**. It ships a live NIDS dashboard with animated charts, CSV batch analysis, an alert investigation drawer, and a streaming AI assistant.

4.1 FRAMEWORK & BUILD TOOLING

Technology	Version	Role in TideWatch
React	19.3	UI library — hooks, contexts, concurrent rendering for the live dashboard
React DOM	19.3	DOM renderer (`src/main.tsx` mount point)
TypeScript	~6.0	Static typing across pages, hooks, API layer, and shared types
Vite	8.3	Dev server + production bundler (`tsc -b && vite build`)
@vitejs/plugin-react	6.1	Fast Refresh & JSX/TSX transform
ESLint	10 (flat config)	Linting with `typescript-eslint` , react-hooks, react-refresh plugins

4.2 STYLING & UI

Technology	Version	Role in TideWatch
Tailwind CSS	4.3 (v4)	Utility-first styling via the native `@tailwindcss/vite` plugin — no config file needed
tailwind-merge	3.7	Conflict-free Tailwind class merging
clsx	2.1	Conditional className construction
Lucide React	1.59	Icon set used across navbar, cards, tables
Custom design system	—	`src/index.css` : Fraunces/DM Sans/JetBrains Mono fonts, green palette, animated SVG waves

4.3 ROUTING, DATA & VISUALIZATION

Technology	Version	Role in TideWatch
React Router	7.18 (v7)	Route tree: Landing, Dashboard, Analyze, Alerts, Assistant, Model, 404
TanStack Query	5.104 (v5)	Server-state caching, background refetch, loading/error states for all API calls
Recharts	3.10 (v3)	Charts: traffic time-series, attack donut, top-IP bars, confusion matrix, feature importance
Papa Parse	5.7	Client-side CSV parsing for drag-and-drop batch analysis & export

4.4 FRONTEND ARCHITECTURE HIGHLIGHTS

- **Typed API layer** — `src/lib/api.ts` dispatches to the FastAPI backend when `VITE_API_URL` is set, and transparently **falls back to local mock data** when the backend is unreachable.
- **Real-time hook** — `src/hooks/useLiveFeed.ts` consumes SSE/WebSocket streams with a simulation fallback.
- **Traffic simulator** — `src/lib/simulation.ts` generates high-fidelity mock flows with 1x/2x/5x speed multipliers and a "Inject Threat" trigger.
- **Path alias** — `@` resolves to `./src` (configured in both `vite.config.ts` and `tsconfig.app.json`).
- **Three tsconfig projects** — `tsconfig.json` solution file referencing `app` (src) and `node` (vite config) configs.

Frontend Folder Map

```
src/
├── components/
│   ├── layout/     # AppLayout · Navbar · BackgroundWaves · Footer
│   ├── charts/     # TrafficLine · AttackDonut · TopIps · ConfusionMatrix · SHAP
│   ├── alerts/     # AlertDrawer (5-tuple + AI SecOps triage report)
│   └── common/     # Drawer · StatCard · Skeleton · SeverityBadge · Markdown
├── pages/          # Landing · Dashboard · Analyze · Alerts · Assistant · Model · 404
├── hooks/          # useCountUp · useLiveFeed · useSimulation
├── lib/            # api.ts · mockData.ts · simulation.ts · utils.ts
├── types/index.ts  # Flow, Alert, ModelMetrics, AiReport contracts
├── App.tsx         # QueryClientProvider + route tree
├── index.css       # design tokens, typography, wave keyframes
└── main.tsx        # entrypoint
```

5. How Frontend & Backend Connect

The two apps are fully decoupled: the frontend only talks to the backend through `src/lib/api.ts` and `useLiveFeed`. Configuration flows through Vite environment variables; CORS is handled by FastAPI middleware.

Step 1 — Frontend env (`.env` in project root)

```
# URL of your FastAPI backend service
VITE_API_URL=http://localhost:8000/api/v1

# Real-time endpoints (Server-Sent Events)
VITE_SSE_URL=http://localhost:8000/api/v1/stream/traffic
```

Step 2 — Backend CORS (`.env`)

```
ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
```

Flow of a single dashboard load

```
Browser ──► Vite (:5173) ──► React DashboardPage
   │                              │
   │                              ▼ TanStack Query
   │                         src/lib/api.ts
   │                              │  (VITE_API_URL set?)
   │                     ┌────────┴─────────┐
   │                     ▼ yes              ▼ no / unreachable
   │            FastAPI :8000          local mock simulation
   │                     │
   ▼                     ▼
SSE ◄── /api/v1/stream/traffic (live flow replay)
   ▲
   └── POST /api/v1/chat (token stream from ReAct agent)
```

> **Graceful degradation:** with no `VITE_API_URL` configured, the frontend runs entirely on built-in mock CICIDS2017 data — so the UI can be demoed without the backend or a database at all.

6. Commands to Run the Project

6.1 FRONTEND (RUN IN PROJECT ROOT `E:\TIDEWATCH`)

```
# 1. Install Node.js dependencies
npm install

# 2. Start the Vite dev server → http://localhost:5173/
npm run dev

# 3. Production build (type-checks with tsc, then bundles)
npm run build

# 4. Preview the production build locally
npm run preview

# 5. Lint the codebase (ESLint flat config)
npm run lint
```

6.2 BACKEND (PYTHON / FASTAPI)

```
# 1. Create and activate a virtual environment
python -m venv .venv

# Windows (PowerShell / CMD)
.venv\Scripts\Activate.ps1   # PowerShell
.venv\Scripts\activate        # CMD

# Linux / macOS
source .venv/bin/activate

# 2. Install Python dependencies
pip install -r requirements.txt

# 3. Create environment config from the template
copy .env.example .env        # Windows
# cp .env.example .env         # Linux/macOS
# → then edit .env: DATABASE_URL, GROQ_API_KEY (optional), ALLOWED_ORIGINS

# 4. Train the ML model (generates artifacts/*.joblib, metrics, samples)
python ml/train.py
# (Runs automatically at API startup if artifacts/model.joblib is missing)

# 5. Apply database migrations
alembic upgrade head

# 6. Start the API server → http://localhost:8000  (docs at /docs)
uvicorn app.main:app --reload --port 8000
```

6.3 RUNNING BACKEND TESTS

```
# pytest suite: predictor, batch endpoints, agent fallbacks, health
# Runs on in-memory SQLite — no live DB or API key required
pytest tests/ -v
```

6.4 DOCKER (FULL STACK)

```
# Build and start API (:8000) + Frontend (:5173)
docker compose up --build

# Optional: include the local Ollama LLM service (:11434)
docker compose --profile ollama up --build
# then pull a model:  docker exec -it tidewatch-ollama ollama pull llama3.1:8b
# and set LLM_PROVIDER=ollama in .env
```

> **What Docker does for you:** the `Dockerfile` runs `python ml/train.py` during build (image ships with pre-generated artifacts), and executes `alembic upgrade head` at container startup before Uvicorn launches with 2 workers.

6.5 QUICK-START CHEAT SHEET (BOTH APPS)

```
# Terminal 1 — Backend
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
python ml/train.py
alembic upgrade head
uvicorn app.main:app --reload --port 8000

# Terminal 2 — Frontend
npm install
npm run dev          # → http://localhost:5173/
```

7. API Endpoint Reference

All routes are prefixed with `/api/v1` (base URL `http://localhost:8000`).

Method	Path	Description
GET	`/health`	Health check: DB ping, ML engine, LLM provider status
POST	`/api/v1/predict`	Single network-flow classification
POST	`/api/v1/predict/batch`	Batch classification of an uploaded CSV
GET	`/api/v1/flows`	Paginated flow records (`?limit&offset`)
GET	`/api/v1/flows/{id}`	Single flow detail
GET	`/api/v1/alerts`	Filterable alert list (`?status&severity`)
GET	`/api/v1/alerts/{id}`	Alert with full AI triage report
PATCH	`/api/v1/alerts/{id}/status`	Update incident status (open/investigating/resolved)
POST	`/api/v1/alerts/{id}/analyze`	Trigger LangGraph triage workflow
GET	`/api/v1/stats/overview`	Aggregate KPI summary
GET	`/api/v1/stats/timeseries`	Normal vs attack traffic time series
GET	`/api/v1/stats/top-sources`	Top adversary source IPs
GET	`/api/v1/stats/attack-distribution`	Attack class proportions
GET	`/api/v1/model/metrics`	Confusion matrix, F1, SHAP/Gini feature importances
GET	`/api/v1/stream/traffic`	SSE: live flow replay stream
POST	`/api/v1/chat`	SSE: token-streaming chat with ReAct agent
GET	`/docs`	Interactive Swagger UI (auto-generated)

8. Environment Variables

Variable	Default	Description
`DATABASE_URL`	`sqlite+aiosqlite:///./tidewatch.db`	Neon pooled Postgres URL or local SQLite
`LLM_PROVIDER`	`groq`	One of: `groq` · `ollama` · `huggingface`
`GROQ_API_KEY`	*empty*	Optional — rule-based fallbacks activate if unset
`GROQ_MODEL`	`llama-3.3-70b-versatile`	Groq model name
`OLLAMA_BASE_URL` / `OLLAMA_MODEL`	`localhost:11434` / `llama3.1:8b`	Local Ollama provider
`HUGGINGFACEHUB_API_TOKEN` / `HF_MODEL`	*empty* / `Qwen/Qwen2.5-7B-Instruct`	HuggingFace provider
`ALLOWED_ORIGINS`	`http://localhost:5173,…`	Comma-separated CORS origins
`ENV`	`development`	`production` disables runtime provider switch
`MODEL_ARTIFACTS_DIR`	`artifacts`	Where ML artifacts are stored
`LOG_LEVEL`	`INFO`	Structured JSON log level
`VITE_API_URL`	*unset*	Frontend → backend base URL; unset = mock mode
`VITE_SSE_URL`	*unset*	Frontend live-traffic SSE endpoint